SnapKitty Red Book — Math Explained Simply

Math for non-technical readers · SnapKitty Collective · 2026

1. Executive Summary

1. SnapKitty is a trust architecture for agents: an agent should only act when a rule, proof, receipt, or oracle gives evidence.

2. The math stack is built around one repeating pattern: make a claim, verify the claim, seal the result, and keep the seal forever.

3. The golden ratio ϕ appears as a weighting and reading system; true contraction uses `1/ϕ`, while ϕ itself is expansion.

4. Fibonacci and `Q(√5)` algebra give the system an exact symbolic way to compute resonance values instead of relying only on floats.

5. WORM seals turn computations into tamper-evident records: if the payload changes, the hash no longer matches.

6. Linear types and resource rules say that certain things, like capabilities or qubits, must be used exactly once.

7. Datalog and Prolog act like law engines: they accept or reject actions according to explicit rules.

8. Lean 4, Coq, and Idris 2 provide proof layers for selected invariants, but they do not prove every marketing claim or every real-world behavior.

9. PIRTM and UTQC use prime indexes, tensor/circuit IR, and receipts to make mathematical computation auditable across languages.

10. The strongest SnapKitty idea is not one theorem by itself; it is the combination of formal checks, runtime receipts, append-only memory, and agent governance.

2. The Big Idea

SnapKitty as a Trust System

SnapKitty treats trust as something that must be produced, not assumed. A message, model output, package install, capability transfer, circuit compilation, or agent action should pass through a gate. The gate may be a Lean theorem, Coq extraction, Prolog rule, Datalog invariant, WORM receipt, hash check, or BOB verdict. If the gate cannot produce evidence, the action should stop.

SnapKitty as a Proof System

The proof system is layered. Lean 4 proves small mathematical and policy facts. Coq proves a verdict algebra and can extract executable OCaml. Idris 2 models no-cloning and one-use resources through Quantitative Type Theory. Datalog proves monotone policy reachability and admissibility. Rust, C++, Haskell, Clojure, APL, and JavaScript then implement executable versions of those ideas.

SnapKitty as an Agent Operating System

An agent operating system needs memory, permissions, process rules, audit logs, and runtime decisions. SnapKitty models those as WORM chains, capabilities, Trust Deeds, BOB verdicts, NATS events, and sealed receipts. The goal is that agents do not just "think"; they leave verifiable traces of what they did and why they were allowed to do it.

SnapKitty as a Mathematical Language

The stack uses several mathematical languages at once. ϕ, Fibonacci, and Q(√5) express resonance and symbolic totals. Linear logic expresses one-use resources. Prime indexing expresses structure and addressing. Datalog and Prolog express law. Lean, Coq, and Idris express proof. WORM seals express memory.

SnapKitty as a Sealed Memory Machine

The WORM rule is simple: once something important happens, seal it with a hash and append it to a chain. This does not prove the event was morally correct or empirically true. It proves that the recorded payload is the same payload later, and that a later reader can detect tampering.

3. The Math Map

Concept	Repo/File	Plain Meaning	Verified By	Status
Golden ratio ϕ	`C:\Users\jessi\Desktop\fibonacci-contraction\lean\FibCore\PhinaryContraction.lean`	A special number where `ϕ² = ϕ + 1`; used as a resonance weight.	Lean theorem `phi_recursion`, `phi_gt_one`	Established classical math implemented
`1/ϕ` contraction	`C:\Users\jessi\IdeaProjects\SNAPKITTYWEST\bob-reasoning-engine\lean\metatron\GrandUnified.lean`	Multiplying by `1/ϕ` shrinks toward a fixed point.	Lean fixed-point definitions for Analysis/Metatron	Established math used architecturally
Fibonacci contraction	`C:\Users\jessi\Desktop\fibonacci-contraction\lean\FibCore\PhinaryContraction.lean`	Fibonacci ratios approach ϕ; actual multiplication by ϕ expands.	Lean ratio theorem; comments mark contraction nuance	Mixed: classical math plus correction
`Q(√5)` algebra	`C:\Users\jessi\IdeaProjects\SNAPKITTYWEST\sovereign-clojure\src\sovereign\resonance.clj`	Every ϕ power is represented exactly as `aϕ + b`.	Clojure symbolic computation	Implementation verification
Galois shadow operator	`sovereign-clojure/src/sovereign/resonance.clj`	Replace ϕ with its conjugate, written in code as `-1/ϕ`, equivalent to `1 - ϕ`.	Clojure function `galois-conjugate`	Established algebra implemented
Norm as meeting point	`sovereign-clojure/src/sovereign/resonance.clj`	Multiply a value by its shadow so the irrational parts cancel into a rational value.	Clojure `trs-norm`	Established algebra used architecturally
Total Resonance Sum / TRS	`sovereign-clojure/src/sovereign/resonance.clj`; `ResonancePipeline.lean`	Sum of symbol biases across pipeline depths.	Clojure exact report; Lean pipeline shapes	SnapKitty-original model
METATRON bidirectional reading	`bob-reasoning-engine/lean/ResonancePipeline.lean`; `metatron/GrandUnified.lean`	A second reading path through the pipeline that can inspect or bypass normal reasoning.	Lean pipeline length/depth/topology checks	SnapKitty-original architecture
ERRANT linear resources	`C:\Users\jessi\IdeaProjects\SNAPKITTYWEST\errant\typing.pl`	Linear items cannot be duplicated; capabilities must be consumed lawfully.	Prolog type rules	Implementation verification
Capabilities as lawful messages	`errant/typing.pl`; `C:\Users\jessi\Desktop\bobs control repo\proofs\datalog\cap_transfer.dl`	A permission is a typed object that authorizes sending, sealing, signing, or transferring.	Prolog and Souffle Datalog rules	Implementation verification
WORM seals	`sovereign-llm/crates/seal/src/lib.rs`; `sovereign-utqc/.../utqc-worm/src/lib.rs`; `snapkitty-chain/src/chain.mjs`	Hash a payload and keep it in an append-only chain.	Rust/JS tests and SHA-256 recomputation	Implementation verification
Datalog admissibility pipeline	`errant/datalog/DatalogEngine.hs`; `proofs/datalog/system_invariants.dl`	A fixed-stage validator that derives accept/reject facts.	Haskell/Datalog rules	Implementation verification
PIRTM multiplicity and receipts	`C:\Users\jessi\IdeaProjects\SNAPKITTYWEST\sovereign-pirtm\src\*.rs`	Prime-indexed tensor computations with WORM-sealed outputs.	Rust tests/utilities	SnapKitty-original architecture
Prime-indexed structures	`sovereign-pirtm/src/prime.rs`; `sovereign-utqc/.../sovereign-prime-mask/src/lib.rs`	Use primes as stable indexes, masks, or tensor coordinates.	Rust primality/mask functions and tests	Established math implemented
Fock space / prime occupation states	Red Book PDF; UTQC prime additions	Model prime modes as occupied/unoccupied states.	Source references in Red Book; limited local crate evidence	Experimental model unless repo source is present

Concept	Repo/File	Plain Meaning	Verified By	Status
SUBLEQ lawful gate	`errant/soul-spec.errant`; Red Book references	A minimal instruction gate used as a lawful computation metaphor.	ERRANT spec references	Experimental/architectural model
UTQC linear qubit safety	`sovereign-utqc/.../utqc-linear/src/lib.rs`	A qubit target can be consumed once; repeated use is rejected.	Rust resource tracker tests	Implementation verification
Sovereign LLM sealed weights	`C:\Users\jessi\IdeaProjects\SNAPKITTYWEST\sovereign-llm\crates\seal\src\lib.rs`	Model weights are hashed so tampering can be detected.	Rust tests: tamper fails, deterministic seal passes	Implementation verification
STELLA zero-knowledge trust receipts	`C:\Users\jessi\Desktop\snapkitty-chain` historical working tree; Red Book references	Proposed pattern for proving control of a private fingerprint without revealing the secret.	No active chain anchoring after cleanup	Prototype / not submitted / not anchored
Lean 4 proof layer	`TrustKernel.lean`, `ResonancePipeline.lean`, `SovereignJudge.lean`	Proves selected definitions and policy facts.	Lean theorem checker where no `sorry` is used	Formal proof layer with scope limits
Coq proof layer	`C:\Users\jessi\Desktop\bobs control repo\proofs\coq\SovereignJudge.v`	Proves verdict algebra and can extract OCaml.	Coq theorem checker and extraction	Formal proof layer
Idris 2 no-cloning layer	`proofs/idris2/SovereignNoCloning.idr`	Linear quantum temp values cannot be cloned by type signature.	Idris 2 QTT model; some trusted `believe_me` use for supplied seal	Formal/experimental bridge
APL executable math layer	`bob-reasoning-engine/apl/*.apl`; `fibonacci-contraction/apl/phinary.apl`	Dense executable math for resonance, Goldilocks, and METATRON calculations.	Runtime execution and comparison to other layers	Executable math layer
Prolog law/type-checking layer	`errant/typing.pl`	Logic rules define which stack programs and capabilities are legal.	Prolog rule success/failure	Implementation verification

4. Theorems and Invariants

4.1 Golden Ratio Recursion

Name: `phi_recursion`

Repo/File: `C:\Users\jessi\Desktop\fibonacci-contraction\lean\FibCore\PhinaryContraction.lean`

Formal Statement: `ϕ ^ 2 = ϕ + 1`

Plain-English Explanation: The golden ratio is the number that turns a square into "itself plus one." This is the core identity behind Fibonacci ratios and phinary notation.

Why It Matters: It lets the system rewrite higher powers of ϕ into simpler expressions.

What the Code Verifies: Lean proves the algebraic identity from the definition `(1 + sqrt 5) / 2`.

What It Does NOT Prove: It does not prove any agent is trustworthy, any model is correct, or any unresolved Fibonacci-prime claim.

Status: Established classical math.

Analogy: It is like a master key for simplifying every golden-ratio expression.

4.2 True Contraction Uses $1/\phi$

Name: Phi contraction correction

Repo/File: `C:\Users\jessi\Desktop\fibonacci-contraction\lean\FibCore\PhinaryContraction.lean`; `C:\Users\jessi\IdeaProjects\SNAPKITTYWEST\bob-reasoning-engine\lean\metatron\GrandUnified.lean`

Formal Statement: The Fibonacci file explicitly notes that `κ = ϕ > 1` expands; contraction requires `κ < 1`. `GrandUnified.lean` uses `PHI_INV := 1 / PHI`.

Plain-English Explanation: Multiplying by ϕ makes a value larger. Multiplying by 1/ϕ makes it smaller.

Why It Matters: This prevents the paper from overstating "contraction" where the implementation is actually an expanding resonance weight.

What the Code Verifies: Lean verifies ϕ > 1, and other Lean files use 1/ϕ for fixed-point/contraction-style operators.

What It Does NOT Prove: It does not prove global convergence of every SnapKitty pipeline.

Status: Established math plus implementation clarification.

Analogy: ϕ is the gas pedal; 1/ϕ is the brake.

4.3 Fibonacci Prime Boundary

Name: fibonacci_primes_infinite

Repo/File: C:\Users\jessi\Desktop\fibonacci-contraction\lean\FibCore\PhinaryContraction.lean

Formal Statement: n, p > n, Nat.Prime p k, fib k = p

Plain-English Explanation: This says there are infinitely many Fibonacci numbers that are prime.

Why It Matters: It marks the edge between proved implementation and open number theory.

What the Code Verifies: Nothing downstream can turn this axiom into a solved theorem; the file honestly labels it open.

What It Does NOT Prove: It does not prove infinitely many Fibonacci primes. That remains an unsolved mathematical question.

Status: Conjectural / experimental model.

Analogy: The road is paved up to a cliff; the axiom is a bridge drawn on the map, not a bridge already built.

4.4 Q(√5) Exact TRS Algebra

Name: Symbolic TRS over Q(√5)

Repo/File: C:\Users\jessi\IdeaProjects\SNAPKITTYWEST\sovereign-clojure\src\sovereign\resonance.clj

Formal Statement: $\phi^n = F(n)\phi + F(n-1)$ and TRS = Aϕ + B

Plain-English Explanation: Instead of storing resonance as only a decimal, the code stores it as an exact golden-ratio expression.

Why It Matters: Exact symbolic values are easier to audit than floating-point approximations.

What the Code Verifies: Clojure computes the coefficients, numeric value, Galois shadow, and norm.

What It Does NOT Prove: It does not prove the chosen biases are objectively correct; it only computes the model exactly.

Status: Established algebra used in a SnapKitty-original model.

Analogy: It is the difference between keeping a recipe and keeping only a photo of the cake.

4.5 Galois Shadow and Norm

Name: Galois shadow operator and rational norm

Repo/File: C:\Users\jessi\IdeaProjects\SNAPKITTYWEST\sovereign-clojure\src\sovereign\resonance.clj

Formal Statement: Code uses σ(ϕ) = -1/ϕ, which equals 1 - ϕ; N(Aϕ + B) = (Aϕ + B)(Aσ(ϕ) + B).

Plain-English Explanation: The shadow replaces the golden ratio with its paired conjugate. Multiplying a value by its shadow cancels the irrational part.

Why It Matters: This gives the architecture a clean way to compare recursive and shadow readings through a rational checkpoint.

What the Code Verifies: The implementation evaluates the shadow and norm for TRS.

What It Does NOT Prove: It does not prove that every "shadow" interpretation in the prose is mathematically forced.

Status: Established algebra with SnapKitty interpretation.

Analogy: It is like checking a reflection against the original and finding the point where both agree.

4.6 METATRON Pipeline Shape

Name: `metatron_depth`, `metatronPipeline_length`, `metatronTopo_valid`

Repo/File: `C:\Users\jessi\IdeaProjects\SNAPKITTYWEST\bob-reasoning-engine\lean\ResonancePipeline.lean`

Formal Statement: METATRON is inserted at depth 5; the injected pipeline has 8 nodes; the topological order has matching length.

Plain-English Explanation: The pipeline has a normal reasoning path and an inserted METATRON path.

Why It Matters: This makes the architecture inspectable as a graph instead of only a story.

What the Code Verifies: Lean checks the declared node count, depth, and ordering facts.

What It Does NOT Prove: It does not prove METATRON has supernatural insight or that every downstream agent answer is correct.

Status: SnapKitty-original architecture with formal shape checks.

Analogy: It proves the subway map has the station where the diagram says it is; it does not prove every passenger reaches the right destination.

4.7 Trust Kernel Biconditional

Name: `resonance_iff_sovereignty`

Repo/File: `C:\Users\jessi\IdeaProjects\SNAPKITTYWEST\bob-reasoning-engine\lean\TrustKernel.lean`

Formal Statement: `resonant deed = true ↔ sovereign deed`

Plain-English Explanation: In this model, a deed is sovereign exactly when it passes both checks: non-empty trusted URI and verified flag.

Why It Matters: It ties a boolean runtime gate to a proof-level definition.

What the Code Verifies: Lean proves both directions and extracts both conditions from one `AND` gate.

What It Does NOT Prove: It does not prove the URI points to a legally valid document or that the verification process was honest outside this model.

Status: Formal proof within defined assumptions.

Analogy: It proves a door opens only when both locks are turned.

4.8 Sovereign Judge Verdict Algebra

Name: Verdict completeness and priority

Repo/File: `C:\Users\jessi\Desktop\bobs control repo\proofs\lean4\SovereignJudge.lean`; `C:\Users\jessi\Desktop\bobs control repo\proofs\coq\SovereignJudge.v`

Formal Statement: Verdicts include approve, reject, defer, escalate, and human-required; priority is bounded; repent escalates.

Plain-English Explanation: Decisions are not free-form text. They belong to a small controlled algebra.

Why It Matters: A controlled verdict type prevents agents from inventing new action states.

What the Code Verifies: Lean and Coq prove selected properties such as approved actions are lawful, repent escalates, and priority is bounded.

What It Does NOT Prove: It does not prove the input evidence is true; it proves the verdict machinery behaves as specified.

Status: Formal proof layer.

Analogy: It is a court form with fixed boxes, not a blank page.

4.9 Coq-to-OCaml Extraction

Name: Certified executable verdict kernel

Repo/File: `C:\Users\jessi\Desktop\bobs control repo\proofs\coq\SovereignJudge.v`

Formal Statement: The Coq file extracts `judge`, `lawful`, `priority`, `combine`, and `requires_human_gate` to OCaml.

Plain-English Explanation: The same logic that was proved can be turned into executable code.

Why It Matters: It narrows the gap between proof and runtime.

What the Code Verifies: Coq verifies the stated theorems before extraction.

What It Does NOT Prove: It does not prove the extracted code is deployed everywhere BOB runs, unless the build/deploy pipeline enforces that.

Status: Formal proof plus certified extraction path.

Analogy: It is like printing a lock from the same blueprint that passed inspection.

4.10 ERRANT Linear Resource Invariant

Name: `valid_errant_image`

Repo/File: `C:\Users\jessi\IdeaProjects\SNAPKITTYWEST\errant\typing.pl`

Formal Statement: `valid_errant_image(Program) :- check_program(Program, [], omega).`

Plain-English Explanation: A program is valid only if it starts empty, follows the typing rules, and ends sealed.

Why It Matters: This makes resources and capabilities explicit instead of hidden in code comments.

What the Code Verifies: Prolog checks stack transitions, linear use, capability requirements, and final sealed halt state.

What It Does NOT Prove: It does not prove the program's business purpose is good or that the hash function itself is collision-proof in a formal cryptographic proof.

Status: Implementation verification.

Analogy: It is a cashier till where every token must be accounted for before closing.

4.11 Cap Transfer Datalog Invariant

Name: `transfer_allowed`, `delegate_allowed`, `seal_allowed`, `jit_allowed`

Repo/File: `C:\Users\jessi\Desktop\bobs control repo\proofs\datalog\cap_transfer.dl`

Formal Statement: Transfers require trust threshold, active cap, ownership, rights bits, WORM-sealed cap, WORM-sealed policy, and no blocked pair.

Plain-English Explanation: A capability can move only when the agent owns it, the policy allows it, and the cap has not been revoked.

Why It Matters: Capability movement is one of the highest-risk parts of an agent system.

What the Code Verifies: Souffle Datalog derives allowed actions from explicit facts and thresholds.

What It Does NOT Prove: It does not prove the source facts are honest; it only proves the derived result follows from them.

Status: Implementation verification.

Analogy: It is a building access system that checks badge, role, policy, and revocation list before unlocking the door.

4.12 Datalog Admissibility Pipeline

Name: 8-stage admissibility validator

Repo/File: `C:\Users\jessi\IdeaProjects\SNAPKITTYWEST\errant\datalog\DatalogEngine.hs`

Formal Statement: artifact -> json_admissible -> nfc_ok -> canonical -> sha256 -> snap_address -> accepted -> worm_receipt.

Plain-English Explanation: Data must pass through a fixed checklist before receiving an address and receipt.

Why It Matters: Deterministic intake prevents "almost valid" artifacts from entering memory.

What the Code Verifies: The Haskell engine derives facts in a fixed order and emits accept/reject state.

What It Does NOT Prove: The local NFC/canonicalization implementation is simplified in this file; production canonicalization needs stricter standards if external interoperability is required.

Status: Implementation verification / scaffold.

Analogy: It is airport security for data.

4.13 WORM Chain Integrity

Name: Append-only seal chain

Repo/File:
`C:\Users\jessi\IdeaProjects\SNAPKITTYWEST\sovereign-utqc\sovereign-utqc\crates\utqc-worm\src\lib.rs`;
`C:\Users\jessi\Desktop\snapkitty-chain\src\chain.mjs`

Formal Statement: Each seal is SHA-256 over a payload and, for chained seals, includes the previous hash.

Plain-English Explanation: Each record points back to the previous record, so tampering breaks the chain.

Why It Matters: This gives agents durable memory and audit trails.

What the Code Verifies: Tests check hash length, append behavior, and chain verification shape.

What It Does NOT Prove: A hash chain does not make bad data good; it only makes later changes detectable.

Status: Implementation verification.

Analogy: It is a notebook where every page contains a fingerprint of the page before it.

4.14 PIRTM Prime-Indexed Tensor Math

Name: Prime-Indexed Recursive Tensor Mathematics

Repo/File: `C:\Users\jessi\IdeaProjects\SNAPKITTYWEST\sovereign-pirtm\src\lib.rs`; `prime.rs`; `tensor.rs`; `seal.rs`

Formal Statement: Tensors are indexed by primes; computations produce WORM seals.

Plain-English Explanation: PIRTM uses prime numbers as stable labels for tensor structure and computation receipts.

Why It Matters: Prime indexes make collisions and ambiguity easier to reason about than arbitrary labels.

What the Code Verifies: Rust implements primality utilities, tensor structures, evolution, and seals.

What It Does NOT Prove: It does not prove every tensor program is mathematically optimal or physically meaningful.

Status: SnapKitty-original architecture with implementation tests.

Analogy: It is a warehouse where every shelf is labeled by a prime number.

4.15 Prime Mask

Name: `PrimeMask`

Repo/File: `C:\Users\jessi\IdeaProjects\SNAPKITTYWEST\sovereign-utqc\sovereign-utqc\crates\sovereign-prime-mask\src\lib.rs`

Formal Statement: A 64-bit mask marks membership over the first 64 primes.

Plain-English Explanation: Each bit says whether a specific prime is attached.

Why It Matters: It gives fast prime-gated indexing and capability-style selection.

What the Code Verifies: Rust tests verify bitwise behavior such as reconstructing a mask through AND/OR.

What It Does NOT Prove: It does not prove prime masks are the best representation for every problem.

Status: Established math implemented.

Analogy: It is a checklist where every checkbox corresponds to a prime.

4.16 UTQC Linear Qubit Safety

Name: Linear resource check

Repo/File: `C:\Users\jessi\IdeaProjects\SNAPKITTYWEST\sovereign-utqc\sovereign-utqc\crates\utqc-linear\src\lib.rs`

Formal Statement: Linear resources may be used exactly once; affine resources at most once; unrestricted resources can be reused.

Plain-English Explanation: A qubit target cannot be consumed twice.

Why It Matters: Quantum and linear-resource code can silently become invalid if values are copied or reused.

What the Code Verifies: Rust tests accept a valid circuit and reject a repeated target use.

What It Does NOT Prove: It does not prove the circuit implements a correct quantum algorithm; it checks resource usage.

Status: Implementation verification.

Analogy: It is a one-use ticket: once scanned, it cannot be scanned again.

4.17 Quantum Algorithm Scaffold

Name: QFT, Grover, QPE, Shor scaffold

Repo/File: `C:\Users\jessi\IdeaProjects\SNAPKITTYWEST\sovereign-utqc\sovereign-utqc\crates\utqc-quantum\src\lib.rs`

Formal Statement: Circuit builders produce gate IR for common quantum algorithms.

Plain-English Explanation: The code can construct simplified circuit forms for known algorithms.

Why It Matters: It connects UTQC resource rules to actual circuit-building workflows.

What the Code Verifies: The builders produce circuit IR without out-of-bounds errors in tested paths.

What It Does NOT Prove: The Shor and oracle pieces are explicitly simplified placeholders, not production-grade quantum implementations.

Status: Experimental implementation scaffold.

Analogy: It is a flight simulator cockpit, not a certified aircraft.

4.18 Sovereign LLM Weight Seal

Name: Model weight WORM seal

Repo/File: `C:\Users\jessi\IdeaProjects\SNAPKITTYWEST\sovereign-llm\crates\seal\src\lib.rs`

Formal Statement: `hash = SHA256(weights || artifact || timestamp)` with chunk checksum.

Plain-English Explanation: The model's weights can be checked later to detect tampering.

Why It Matters: It gives model artifacts provenance and integrity.

What the Code Verifies: Rust tests confirm tampered weights fail, deterministic inputs match, empty and large weights seal, and save/load works.

What It Does NOT Prove: It does not prove the model is accurate, unbiased, safe, or trained ethically.

Status: Implementation verification.

Analogy: It is a tamper sticker on a machine, not a guarantee that the machine makes correct decisions.

4.19 STELLA ZK Trust Receipt Pattern

Name: Sovereign fingerprint proof pattern

Repo/File: Red Book references; historical `snapkitty-chain` STELLA files were removed from active chain surface after deciding not to submit/anchor to Stellar.

Formal Statement: Intended claim: prove control of a private authorship fingerprint without revealing the private key.

Plain-English Explanation: A user could prove they know a secret linked to a public fingerprint without exposing the secret itself.

Why It Matters: This is useful for private authorship and compliance receipts.

What the Code Verifies: In the current cleaned chain surface, no active Stellar anchoring is claimed.

What It Does NOT Prove: It does not prove a Stellar on-chain verification happened, because that was not submitted or anchored.

Status: Prototype / experimental model.

Analogy: It is showing you have the key to a safe without opening the safe.

4.20 Idris 2 No-Cloning

Name: `noCloningProof`

Repo/File: `C:\Users\jessi\Desktop\bobs control repo\proofs\idris2\SovereignNoCloning.idr`

Formal Statement: `noCloningProof : (1 qt : QuantumTemp) -> ObservationResult`

Plain-English Explanation: The type signature says the quantum temporary value must be used exactly once.

Why It Matters: Some resources should not be copied or discarded in safety-critical systems.

What the Code Verifies: Idris 2's QTT model enforces linear usage at type level for the shown functions.

What It Does NOT Prove: The file uses `believe_me` for an externally supplied seal proof, so that part is trusted rather than fully constructed.

Status: Formal/experimental bridge.

Analogy: It is a library book that must be returned once and cannot be photocopied.

4.21 APL Executable Math

Name: APL resonance and phinary layer

Repo/File: `C:\Users\jessi\IdeaProjects\SNAPKITTYWEST\bob-reasoning-engine\apl\*.apl`; `C:\Users\jessi\Desktop\fibonacci-contraction\apl\phinary.apl`

Formal Statement: Executable array programs compute compact mathematical transforms such as Goldilocks zones, METATRON solving, and phinary calculations.

Plain-English Explanation: APL is the calculator layer: short programs express dense math directly.

Why It Matters: It gives the system an executable math notation that can be compared against Rust, Lean, or Clojure outputs.

What the Code Verifies: Runtime execution can verify numeric agreement and smoke-test formulas.

What It Does NOT Prove: APL execution is not the same as a theorem unless tied to a proof layer.

Status: Executable math layer.

Analogy: It is the lab bench where formulas are run, not the court where formulas are formally proven.

4.22 Prolog Law Layer

Name: Prolog as law/type checker

Repo/File: `C:\Users\jessi\IdeaProjects\SNAPKITTYWEST\errant\typing.pl`

Formal Statement: Operations are valid when their input stack matches the rule and produces the required output stack.

Plain-English Explanation: Prolog defines what legal computation steps look like.

Why It Matters: This makes legal execution declarative and inspectable.

What the Code Verifies: Prolog succeeds for lawful programs and fails for illegal stack/capability flows.

What It Does NOT Prove: It does not prove every interpreter using the rules implements them perfectly.

Status: Implementation verification.

Analogy: It is a rulebook that a referee can apply move by move.

5. What Is Truly Novel

The individual math ingredients are mostly established: ϕ, Fibonacci identities, Galois conjugation, norms, prime numbers, hash chains, Datalog, Prolog, linear logic, and theorem provers all existed before SnapKitty. The novelty is the system composition:

- Mathematical constants and symbolic algebra are used as pipeline weights.
- Formal proofs are paired with runtime agents instead of being left as separate papers.
- WORM seals are applied across LLM weights, circuits, receipts, chain events, and agent decisions.
- Capability transfer, package gates, and agent actions are treated as lawful messages.
- Multiple proof styles coexist: Lean for theorem statements, Coq for extraction, Idris for linear resources, Datalog for monotone policy, Prolog for operational law.
- The architecture is agent-native: source files are written not only for humans, but also as instructions and constraints for future agents.

That combination is the strongest claim: not "we invented the golden ratio," but "we wired classical math, formal methods, audit receipts, and agent governance into one executable trust stack."

6. What This Guide Intentionally Does Not Claim

- It does not claim SnapKitty proves all AI outputs are true.
- It does not claim WORM seals make private user data ethically safe by themselves.
- It does not claim ϕ multiplication is contraction; only $1/\phi$ is contraction.
- It does not claim the open Fibonacci-prime axiom is solved.
- It does not claim experimental STELLA/Stellar anchoring happened after that path was intentionally dropped.
- It does not claim every repo has the same proof strength.
- It does not claim simplified quantum scaffolds are production quantum algorithms.
- It does not claim formal definitions automatically prove real-world legal compliance.

7. Reader's One-Sentence Takeaway

SnapKitty is best understood as a sealed trust machine: math defines the rules, proof tools check selected rules, runtime engines execute the rules, and WORM memory preserves the evidence.